PROYECTO: Analizador Sintáctico para MiniJava usando Pyparsing

1. Objetivo general

Diseñar e implementar un analizador sintáctico (parser) en Python, utilizando la librería Pyparsing, para el lenguaje MiniJava definido en el Laboratorio 1. El analizador debe:

- Validar la estructura gramatical del programa fuente.
- Emplear como terminales los mismos tokens definidos en el analizador léxico del Laboratorio 1.
- Construir una representación estructurada del programa (árbol de análisis sintáctico).
- Detectar y reportar errores sintácticos claros y localizados, sin detener el análisis.

Este laboratorio continúa el anterior: el lenguaje es el mismo MiniJava, de modo que todo token reconocido por el analizador léxico tiene aquí un uso sintáctico. No se repite la especificación léxica; se asume conocida.

2. Alcance

El analizador sintáctico debe soportar los siguientes elementos del lenguaje.

2.1 Declaraciones

Un tipo, un identificador y, opcionalmente, una inicialización.

```
int x = 10;
float y = 3.5 * x;
String msg = "Hola";
boolean activo = true;
char inicial = 'E';
int contador;
```

2.2 Asignaciones

Asignación simple y asignación compuesta con los operadores +=, -=, *= y /=.

```
x = 5;
y = 3 + 7 * x;
z += 2;
w *= a + b;
```

2.3 Incremento y decremento

```
contador++;
contador--;
```

2.4 Expresiones

Se admiten expresiones aritméticas (+, -, *, /, %), relacionales (<, <=, >, >=, ==, !=) y lógicas (&&, ||, !), con identificadores, literales numéricos, cadenas, caracteres y los literales booleanos true y false. Se permiten paréntesis para agrupar.

La precedencia, de mayor a menor, es la siguiente:

Nivel	Operadores	Asociatividad
1 (mayor)	`!  -  (unarios)`	derecha
2	`*  /  %`	izquierda
3	`+  -`	izquierda
4	`<  <=  >  >=  ==  !=`	izquierda
5	`&&`	izquierda
6 (menor)	`||`	izquierda

```
resultado = (x + 2) * 30 % y;
cuenta = 2 * z + 9 - 7 * (k + l);
valido = a > 5 && b < 3;
negado = !activo;
```

2.5 Bloques

Los bloques se delimitan con llaves { y }. Un bloque puede contener cero o más sentencias y puede aparecer anidado dentro de cualquier estructura de control.

2.6 Condicionales (if-else)

La condición va siempre entre paréntesis. El bloque else es opcional y puede contener a su vez otro if, lo que permite cadenas else-if.

```
if (x > 3) {
    y = x + 2;
    z = 2 * y - 9;
}
else {
    y = x - 2;
}

if (x + 1 > 3) {
    y = x + 2;
}

if (a < b) {
    x = 1;
} else if (a == b) {
    x = 2;
} else {
    x = 3;
}
```

2.7 Ciclo while

```
while (a < 5) {
    resultado = a + 1;
    a++;
}

while (x + 3 < 5) {
    resultado = b + 2 * (c + 1);
}
```

2.8 Ciclo do-while

El cuerpo se ejecuta antes de evaluar la condición. La sentencia termina obligatoriamente en punto y coma.

```
do {
    contador--;
    z *= 2;
} while (contador > 0);
```

2.9 Ciclo for

Tiene la forma for (inicio; condición; [paso]) seguida de un bloque. El paso es opcional: si se omite, su valor es 1 cuando la condición es creciente (< o <=) y -1 cuando es decreciente (> o >=).

```
for (int i = 0; i < 7; i++)     { x = i; }
for (int i = 0; i < 7; )        { x = i; }
for (int i = 0; i < 7; i = i + 2) { x = i; }
for (int i = 10; i >= 0; i--)   { x = i; }
for (int i = 10; i >= 0; )      { x = i; }
for (int i = 0; i < 20; i += 5) { x = i; }
```

El inicio declara o asigna una sola variable. La declaración múltiple separada por comas, como for (int i = 0, n = 5; ...), no forma parte de MiniJava y debe reportarse como error de encabezado.

2.10 Selección múltiple (switch-case)

La expresión de control va entre paréntesis. Cada case va seguido de un literal, dos puntos y cero o más sentencias. El bloque default es opcional y, si aparece, va al final.

```
switch (z) {
    case 1:
        System.out.println("uno");
        break;
    case 2:
        System.out.println("dos");
        break;
    default:
        System.out.println("otro");
}
```

2.11 Salida por consola

Se admiten System.out.print y System.out.println, con una expresión como argumento.

```
System.out.println(mensaje);
System.out.print(a + b);
```

2.12 Sentencias de control de flujo

```
break;
continue;
```

2.13 Otras consideraciones

- El programa fuente puede contener comentarios de una línea (//) y multilínea (/* */), según lo definido en el Laboratorio 1. El analizador sintáctico debe ignorarlos.
- Deben tolerarse tabulaciones, espacios en blanco y líneas en blanco en cualquier posición.
- Debe admitirse el anidamiento de cualquier estructura dentro de cualquier otra.

- Las cadenas se delimitan con comillas dobles y los caracteres con comillas simples, tal como en el Laboratorio 1. Las cadenas solo aparecen en asignaciones, comparaciones y argumentos de salida.
- No se requiere definición de clases, métodos, arreglos ni funciones de usuario.
- No se requiere ejecutar el programa: únicamente validar su sintaxis.

3. Formato del árbol de análisis sintáctico

El árbol se representa con listas anidadas entre corchetes. Esta sección define la forma de cada nodo sin ambigüedad: dos implementaciones correctas deben producir exactamente el mismo árbol para el mismo programa.

3.1 Reglas generales

- Un programa es la lista de sus sentencias: [sentencia1, sentencia2, ...].
- Un bloque es, igualmente, la lista de sus sentencias. No se añade ningún nivel adicional de corchetes.
- Todos los operadores, incluidos los lógicos, se escriben en notación infija, en el mismo orden en que aparecen en el programa.
- Los literales conservan su forma textual: las cadenas y los caracteres mantienen sus comillas.

3.2 Expresiones

Una expresión atómica se representa por su propio lexema. Una expresión compuesta se representa por la lista de sus operandos y operadores; los operadores de un mismo nivel de precedencia quedan en una sola lista, y los de distinto nivel se anidan.

Expresión	Representación
`X`	`'x'`
`10`	`'10'`
`x + y`	`['x', '+', 'y']`
`a > 5 && b < 3`	`[['a', '>', '5'], '&&', ['b', '<', '3']]`
`(x + 2) * 30 % y`	`[['x', '+', '2'], '*', '30', '%', 'y']`
`!activo`	`['!', 'activo']`

3.3 Sentencias

En las declaraciones y asignaciones, la expresión se funde con el prefijo del nodo: si la expresión es atómica se añade como un elemento, y si es compuesta se añaden sus elementos.

Sentencia	Nodo del árbol
`int x = 10;`	`['int', 'x', '10']`
`float y = 3.5 * x;`	`['float', 'y', '3.5', '*', 'x']`
`int contador;`	`['int', 'contador']`
`z = 0;`	`['z', '0']`
`z = z + i;`	`['z', 'z', '+', 'i']`

Sentencia	Nodo del árbol
`z += 2;`	`['z', '+=', '2']`
`contador++;`	`['contador', '++']`
`break;`	`['break']`
`continue;`	`['continue']`
`System.out.println(x);`	`['println', 'x']`
`System.out.print(a + b);`	`['print', 'a', '+', 'b']`

3.4 Estructuras de control

Estructura	Nodo del árbol
if sin else	`['if', condición, bloque]`
if con else	`['if', condición, bloque, bloqueElse]`
While	`['while', condición, bloque]`
do-while	`['dowhile', bloque, condición]`
For	`['for', encabezado, bloque]`
switch	`['switch', expresión, [casos]]`
Case	`['case', literal, bloque]`
default	`['default', bloque]`

El encabezado del for tiene la forma [tipo, variable, valorInicial, condición, paso], donde el tipo se omite si la inicialización reutiliza una variable ya declarada. El paso se representa igual que una sentencia de incremento o de asignación.

Encabezado del for	Representación del paso
`for (int i = 0; i < 5; i++)`	`['int','i','0',['i','<','5'],['i','++']]`
`for (int i = 0; i < 5; )`	`['int','i','0',['i','<','5'],['i','1']]`
`for (int i = 10; i >= 0; )`	`['int','i','10',['i','>=','0'],['i','-1']]`
`for (int i = 0; i < 7; i = i + 2)`	`['int','i','0',['i','<','7'],['i','i','+','2']]`
`for (int i = 0; i < 20; i += 5)`	`['int','i','0',['i','<','20'],['i','+=','5']]`

3.5 Nodos de error

Todo error sintáctico se representa con un nodo de exactamente tres elementos:

```
['error', <contexto>, <descripción>]
```

El nodo de error reemplaza al elemento de la estructura que no pudo reconocerse, y el resto de la estructura se conserva. Por ejemplo, si en un while falta el paréntesis de cierre, el nodo de error ocupa el lugar de la condición y el bloque se analiza normalmente:

```
while (x < 20 {
    x++;
}

→  ['while', ['error', 'while', 'falta parentesis de cierre'], [['x', '++']]]
```

4. Errores sintácticos

El analizador no debe detenerse ante el primer error: debe reportarlo, recuperarse y continuar el análisis del resto del programa. La estrategia de recuperación consiste en producciones de error dentro de la gramática, complementadas con una regla de último recurso que descarta texto hasta el siguiente punto y coma o llave de cierre.

Deben reconocerse al menos los siguientes errores:

Contexto	Descripción	Ejemplo
`declaracion`	falta punto y coma	`int x = 10`
`asignacion`	falta punto y coma	`x = 1`
`incremento`	falta punto y coma	`contador++`
`Break`	falta punto y coma	`break`
`continue`	falta punto y coma	`continue`
`Print`	falta punto y coma	`System.out.print(x)`
`Print`	falta parentesis de cierre	`System.out.print(x;`
`condicion`	condicion vacia	`if () {`
`condicion`	expresion incompleta	`if (x > ) {`
`if / while / dowhile / switch`	falta parentesis de cierre	`while (x < 20 {`
`For`	faltan separadores ';' en el encabezado	`for (int i=0; i<3 i++)`
`Case`	faltan dos puntos	`case 1`
`bloque`	falta llave de cierre	`if (x) { y = 1;`
`sentencia`	sentencia no reconocida	`int 456 = 7;`

Cada error se reporta con el formato:

```
ERROR SINTACTICO (linea N): <contexto> - <descripción>
```

Los errores se listan ordenados por número de línea.

5. Salidas del programa

- No se consideran errores léxicos: los programas de prueba están lexicográficamente bien construidos.
- Si el programa es sintácticamente correcto, se imprime Código Válido, seguido del árbol de análisis sintáctico.
- Si el programa tiene errores, se imprime Errores de sintaxis, la lista de errores encontrados y, a continuación, el árbol de análisis sintáctico con los nodos de error en su posición.
- En ambos casos el árbol se imprime con los anidamientos dentro de corchetes, con sangría, tal como muestran los ejemplos de la sección 8.

6. Requisitos de implementación

Debe implementarse usando:

- Python
- Pyparsing

No se permite:

- lex/yacc
- PLY
- ANTLR
- Lark u otro generador de analizadores

El programa debe leer un archivo de texto con cualquier nombre y extensión .txt, cargado de forma interactiva, y analizarlo completo.

6.1 Uso obligatorio de Pyparsing

Deben usarse al menos las siguientes construcciones:

```
Forward, Group, MatchFirst, Optional, ZeroOrMore,
OneOrMore, Suppress, Keyword, Literal, Regex, infix_notation
```

Forward es necesario para la recursión de la gramática (una sentencia contiene bloques, y un bloque contiene sentencias). infix_notation resuelve la precedencia y la asociatividad de los operadores sin construir a mano la cadena expresión → término → factor.

7. Gramática libre de contexto

La celda 2 del entregable debe contener la gramática libre de contexto del lenguaje, escrita en Markdown con LaTeX según las normas descritas en clase. Debe incluir:

- El conjunto de símbolos no terminales y el símbolo inicial.
- El conjunto de símbolos terminales, que corresponde a los tokens del Laboratorio 1.
- Todas las producciones, incluidas las de las estructuras de control y las de la jerarquía de expresiones con su precedencia.
- Una indicación explícita de cómo se resuelve la ambigüedad del else colgante (dangling else).

8. Ejemplos completos

8.1 Programa 1: sin errores

```
int x = 10;
float y = 3.14;
String mensaje = "Hola mundo";

if (x >= 10) {
    System.out.println(mensaje);
}

for (int i = 0; i < 5; i++) {
    if (i < 3) {
        System.out.println(i);
    }
```

```
}

// comentario simple

/*
comentario
multilinea
*/

while (x < 20) {
    x++;
}
```

Salida esperada

```
Codigo Valido

Arbol de Analisis Sintactico

[
  ['int', 'x', '10'],
  ['float', 'y', '3.14'],
  ['String', 'mensaje', '"Hola mundo"'],
  [
    'if',
    ['x', '>=', '10'],
    [
      ['println', 'mensaje']
    ]
  ],
  [
    'for',
    [
      'int',
      'i',
      '0',
      ['i', '<', '5'],
      ['i', '++']
    ],
    [
      [
        'if',
        ['i', '<', '3'],
        [
          ['println', 'i']
        ]
      ]
    ]
  ],
  [
    'while',
    ['x', '<', '20'],
    [
      ['x', '++']
    ]
  ]
]
```

8.2 Programa 2: sin errores, con todas las estructuras

```
int x = 5;
int y = 10;
int z = 0;
boolean activo = true;
char inicial = 'E';
```

```
String nombre = "Juan";

if (x < y) {
    if (y > 5) {
        System.out.println(y);
    } else {
        System.out.print(x);
    }
} else {
    if (x == 5) {
        z = x + y;
    } else {
        z = 0;
    }
}

while (x < 10) {
    for (int i = 0; i < 3; i++) {
        z = z + i;
    }
    x++;
}

do {
    y--;
    z *= 2;
} while (y > 0);

for (int k = 10; k >= 0; ) {
    z -= k;
}

for (int m = 0; m < 20; m = m + 5) {
    z += m;
}

switch (z) {
    case 1:
        System.out.println("uno");
        break;
    case 2:
        System.out.println("dos");
        break;
    default:
        System.out.println("otro");
}

if (nombre == "Juan" && activo || inicial != 'A') {
    z = (x + 2) * 30 % y;
    activo = !activo;
}
```

Salida esperada

```
Codigo Valido


Arbol de Analisis Sintactico


[
  ['int', 'x', '5'],
  ['int', 'y', '10'],
  ['int', 'z', '0'],
  ['boolean', 'activo', 'true'],
  ['char', 'inicial', "'E'"],
```

```
  ['String', 'nombre', '"Juan"'],
  [
    'if',
    ['x', '<', 'y'],
    [
      [
        'if',
        ['y', '>', '5'],
        [
          ['println', 'y']
        ],
        [
          ['print', 'x']
        ]
      ]
    ],
    [
      [
        'if',
        ['x', '==', '5'],
        [
          ['z', 'x', '+', 'y']
        ],
        [
          ['z', '0']
        ]
      ]
    ]
  ],
  [
    'while',
    ['x', '<', '10'],
    [
      [
        'for',
        [
          'int',
          'i',
          '0',
          ['i', '<', '3'],
          ['i', '++']
        ],
        [
          ['z', 'z', '+', 'i']
        ]
      ],
      ['x', '++']
    ]
  ],
  [
    'dowhile',
    [
      ['y', '--'],
      ['z', '*=', '2']
    ],
    ['y', '>', '0']
  ],
  [
    'for',
    [
      'int',
      'k',
      '10',
      ['k', '>=', '0'],
      ['k', '-1']
    ],
    [
```

```
    ['z', '-=', 'k']
  ]
],
[
  'for',
  [
    'int',
    'm',
    '0',
    ['m', '<', '20'],
    ['m', 'm', '+', '5']
  ],
  [
    ['z', '+=', 'm']
  ]
],
[
  'switch',
  'z',
  [
    [
      'case',
      '1',
      [
        ['println', '"uno"'],
        ['break']
      ]
    ],
    [
      'case',
      '2',
      [
        ['println', '"dos"'],
        ['break']
      ]
    ],
    [
      'default',
      [
        ['println', '"otro"']
      ]
    ]
  ]
],
[
  'if',
  [
    [
      ['nombre', '==', '"Juan"'],
      '&&',
      'activo'
    ],
    '||',
    ['inicial', '!=', "'A'"]
  ],
  [
    [
      'z',
      ['x', '+', '2'],
      '*',
      '30',
      '%',
      'y'
    ],
    ['activo', '!', 'activo']
  ]
]
```

```
]
```

8.3 Programa 3: con errores sintácticos

```
int x = 10;
int y = 5;

if (x > ) {
    for (int i = 0; i < 3 i++) {
        y = y + i
    }
} else {
    System.out.println(y);
}

while (x < 20 {
    x++;
}
```

Salida esperada

```
Errores de sintaxis

ERROR SINTACTICO (linea 4): condicion - expresion incompleta
ERROR SINTACTICO (linea 5): for - faltan separadores ';' en el encabezado
ERROR SINTACTICO (linea 6): asignacion - falta punto y coma
ERROR SINTACTICO (linea 12): while - falta parentesis de cierre

Arbol de Analisis Sintactico

[
  ['int', 'x', '10'],
  ['int', 'y', '5'],
  [
    'if',
    ['error', 'condicion', 'expresion incompleta'],
    [
      [
        'for',
        ['error', 'for', "faltan separadores ';' en el encabezado"],
        [
          ['error', 'asignacion', 'falta punto y coma']
        ]
      ]
    ],
    [
      ['println', 'y']
    ]
  ],
  [
    'while',
    ['error', 'while', 'falta parentesis de cierre'],
    [
      ['x', '++']
    ]
  ]
]
```

8.4 Programa 4: con errores sintácticos de todos los tipos

```
String nombre = "Juan";
String estado = "activo"

int contador = 0;
```

```
if (nombre == "Juan" && estado == "activo") {
    System.out.println(nombre);
} else {
    System.out.println("inactivo");
}

if () {
    contador++;
}

if (contador > ) {
    contador--;
}

if (contador > 3 {
    System.out.print(contador)
}

while (nombre != "" || estado == "activo") {
    nombre = nombre + "!";
    estado = "activo"
    System.out.println(estado;
    contador++
}

do {
    contador--
} while (contador > 0)

switch (contador) {
    case 1
        System.out.println("uno");
        break
    case 2:
        continue
    default:
        System.out.println("otro");
}

for (int i = 0; i < 5) {
    contador += i;
}

int 456 = 7;

if (contador > 3) {
    System.out.println("fin");
```

Salida esperada

```
Errores de sintaxis

ERROR SINTACTICO (linea 2): declaracion - falta punto y coma
ERROR SINTACTICO (linea 12): condicion - condicion vacia
ERROR SINTACTICO (linea 16): condicion - expresion incompleta
ERROR SINTACTICO (linea 20): if - falta parentesis de cierre
ERROR SINTACTICO (linea 21): print - falta punto y coma
ERROR SINTACTICO (linea 26): asignacion - falta punto y coma
ERROR SINTACTICO (linea 27): print - falta parentesis de cierre
ERROR SINTACTICO (linea 28): incremento - falta punto y coma
ERROR SINTACTICO (linea 31): dowhile - falta punto y coma
ERROR SINTACTICO (linea 36): case - faltan dos puntos
```

```
ERROR SINTACTICO (linea 38): break - falta punto y coma
ERROR SINTACTICO (linea 40): continue - falta punto y coma
ERROR SINTACTICO (linea 45): for - faltan separadores ';' en el encabezado
ERROR SINTACTICO (linea 49): sentencia - sentencia no reconocida
ERROR SINTACTICO (linea 51): bloque - falta llave de cierre

Arbol de Analisis Sintactico

[
  ['String', 'nombre', '"Juan"'],
  ['error', 'declaracion', 'falta punto y coma'],
  ['int', 'contador', '0'],
  [
    'if',
    [
      ['nombre', '==', '"Juan"'],
      '&&',
      ['estado', '==', '"activo"']
    ],
    [
      ['println', 'nombre']
    ],
    [
      ['println', '"inactivo"']
    ]
  ],
  [
    'if',
    ['error', 'condicion', 'condicion vacia'],
    [
      ['contador', '++']
    ]
  ],
  [
    'if',
    ['error', 'condicion', 'expresion incompleta'],
    [
      ['contador', '--']
    ]
  ],
  [
    'if',
    ['error', 'if', 'falta parentesis de cierre'],
    [
      ['error', 'print', 'falta punto y coma']
    ]
  ],
  [
    'while',
    [
      ['nombre', '!=', '""'],
      '||',
      ['estado', '==', '"activo"']
    ],
    [
      ['nombre', 'nombre', '+', '"!"'],
      ['error', 'asignacion', 'falta punto y coma'],
      ['error', 'print', 'falta parentesis de cierre'],
      ['error', 'incremento', 'falta punto y coma']
    ]
  ],
  ['error', 'dowhile', 'falta punto y coma'],
  [
    'switch',
    'contador',
    [
```

```
      [
        'case',
        ['error', 'case', 'faltan dos puntos'],
        [
          ['println', '"uno"'],
          ['error', 'break', 'falta punto y coma']
        ]
      ],
      [
        'case',
        '2',
        [
          ['error', 'continue', 'falta punto y coma']
        ]
      ],
      [
        'default',
        [
          ['println', '"otro"']
        ]
      ]
    ]
  ],
  [
    'for',
    ['error', 'for', "faltan separadores ';' en el encabezado"],
    [
      ['contador', '+=', 'i']
    ]
  ],
  ['error', 'sentencia', 'sentencia no reconocida'],
  [
    'if',
    ['contador', '>', '3'],
    [
      ['println', '"fin"'],
      ['error', 'bloque', 'falta llave de cierre']
    ]
  ]
]
```

9. Grupos y entregables

El trabajo debe ser desarrollado por los mismos grupos del Laboratorio 1.

Archivo en Python en un cuaderno de Google Colab con nombre **apellido1nombre1-apellido2nombre2-apellido3nombre3-apellido4nombre4.ipynb**. Este archivo debe contener en distintas celdas lo siguiente:

- Celda 1: Título del proyecto, nombres de autores y NRC al que pertenece cada estudiante.
- Celda 2: **Gramática Libre de Contexto** del lenguaje implementado, escrita en texto Markdown con LaTeX según las normas descritas en clase.
- Celda 3: **Manual de uso del programa**, con indicaciones y directrices claras sobre los elementos a considerar para su ejecución, como el cargue de librerías. Debe ser escrito en Markdown.
- Celda 4: **Código en Python** que implemente el analizador sintáctico.

Durante la sustentación se cargarán archivos de prueba distintos a los de este enunciado, por lo que el programa debe funcionar con cualquier archivo .txt que el evaluador seleccione.

10. Entrega y evaluación

Entrega: 13 de noviembre de 2026

Sustentaciones: última semana de clases

Evaluación:

- Pruebas del archivo .ipynb: 60%
    - Gramática libre de contexto
    - Manual
    - Programa correcto
- Sustentación: 40%
    - Selección de un integrante del grupo de forma aleatoria.
    - Cambios en el archivo .ipynb del grupo.
    - Tiempo: Máximo una hora.

11. Preguntas

En el tema ***Preguntas Laboratorio 2 - Análisis Sintáctico*** del Foro en el Brightspace de la asignatura pueden formular sus preguntas.